



National University of Modern Languages, Islamabad

LAB 8

Submitted To:	Sir Shahid
Submitted By:	Tayyaba Ejaz
Roll Number:	2751
Subject:	MAD
Due Date:	15th December 2025
Semester:	8th
Course:	BSCS, Morning

Faculty of Engineering and Computer Science

Table of contents

1. Lab Task for Attendance App.....	3
1.1 Description	3
1.2 Code	3
1.2.1 bloc: alert_bloc.dart	3
1.2.2 bloc: alert_event.dart.....	4
1.2.3 bloc: alert_state.dart.....	5
1.2.4 Model: weather_alert.dart	5
1.2.5 Repository: weather_repository.dart.....	6
1.2.6 bloc: alert_state.dart.....	6
1.2.7 main.dart	7
1.3 Screenshots.....	10

1. Lab Task for Attendance App

A Flutter weather monitoring app shows a screen called “City Alerts”. This screen must fetch a list of active weather alerts (e.g., “Heavy Rain”, “Fog”, “Heatwave”) from a remote API when the user taps a button “Load Alerts”. While fetching, the UI must show a loader. After success, the UI shows the alerts list.

Develop the structure using the Bloc pattern to fetch alerts from an external API:

1. Define the three core AlertState classes:
 - AlertInitial
 - AlertLoading
 - AlertLoaded (must carry a List<WeatherAlert>)
2. Define the FetchAlertsEvent class.
3. Write the complete implementation for an AlertBloc class, including:
 - initial state setup
 - the on<FetchAlertsEvent> handler that performs the async API call and emits states in this order:
Loading → Loaded

Assume a repository exists:

```
class WeatherRepository {  
  Future<List<WeatherAlert>> getActiveAlerts();  
}
```

1.1 Description

This task implements the **City Alerts** feature using the **Bloc (Business Logic Component) pattern** in Flutter. The purpose is to manage asynchronous data fetching from a remote weather API while keeping the user interface responsive and organized. When the user taps the “**Load Alerts**” button, the Bloc handles the request by transitioning through defined states: **Initial**, **Loading**, and **Loaded**. During the loading state, a progress indicator is displayed, and upon successful data retrieval, a list of active weather alerts is shown on the screen. This approach ensures a clear separation between UI and business logic, improves maintainability, and provides efficient state management.

1.2 Code

1.2.1 bloc: alert_bloc.dart

```
import 'package:flutter_bloc/flutter_bloc.dart';
```

```

import 'alert_event.dart';
import 'alert_state.dart';
import '../repository/weather_repository.dart';

class AlertBloc extends Bloc<AlertEvent, AlertState> {
  final WeatherRepository repository;

  AlertBloc(this.repository) : super(AlertInitial()) {
    on<FetchAlertsEvent>((event, emit) async {
      // Emit Loading
      emit(AlertLoading());

      // Fetch data from API
      final alerts = await repository.getActiveAlerts();

      // Emit Loaded
      emit(AlertLoaded(alerts));
    });
  }
}

```

1.2.2 bloc: alert_event.dart

```

import 'package:equatable/equatable.dart';

abstract class AlertEvent extends Equatable {
  @override
  List<Object?> get props => [];
}

class FetchAlertsEvent extends AlertEvent {}

```

1.2.3 **bloc: alert_state.dart**

```
import 'package:equatable/equatable.dart';
import '../models/weather_alert.dart';

abstract class AlertState extends Equatable {
  @override
  List<Object?> get props => [];
}

// Initial State
class AlertInitial extends AlertState {}

// Loading State
class AlertLoading extends AlertState {}

// Loaded State
class AlertLoaded extends AlertState {
  final List<WeatherAlert> alerts;

  AlertLoaded(this.alerts);

  @override
  List<Object?> get props => [alerts];
}
```

1.2.4 **Model: weather_alert.dart**

```
class WeatherAlert {
  final String title;

  WeatherAlert({required this.title});
}
```

```
}
```

1.2.5 Repository: `weather_repository.dart`

```
import '../models/weather_alert.dart';
```

```
class WeatherRepository {  
  Future<List<WeatherAlert>> getActiveAlerts() async {  
    // simulate network delay  
    await Future.delayed(const Duration(seconds: 2));  
  
    return [  
      WeatherAlert(title: "Heavy Rain"),  
      WeatherAlert(title: "Fog"),  
      WeatherAlert(title: "Heatwave"),  
    ];  
  }  
}
```

1.2.6 bloc: `alert_state.dart`

```
import 'package:equatable/equatable.dart';
```

```
import '../models/weather_alert.dart';
```

```
abstract class AlertState extends Equatable {  
  @override  
  List<Object?> get props => [];  
}
```

```
// Initial State
```

```
class AlertInitial extends AlertState {}
```

```
// Loading State
```

```
class AlertLoading extends AlertState {}

// Loaded State
class AlertLoaded extends AlertState {
  final List<WeatherAlert> alerts;

  AlertLoaded(this.alerts);

  @override
  List<Object?> get props => [alerts];
}
```

1.2.7 main.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'bloc/alert_bloc.dart';
import 'bloc/alert_event.dart';
import 'bloc/alert_state.dart';
import 'repository/weather_repository.dart';

void main() {
  runApp(MyApp());
}

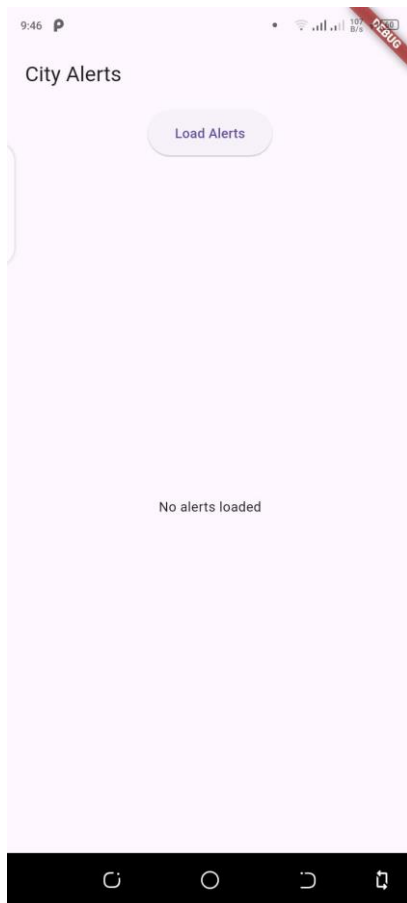
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
```

```
return MaterialApp(  
  home: BlocProvider(  
    create: (_) => AlertBloc(WeatherRepository()),  
    child: CityAlertsScreen(),  
  ),  
);  
}
```

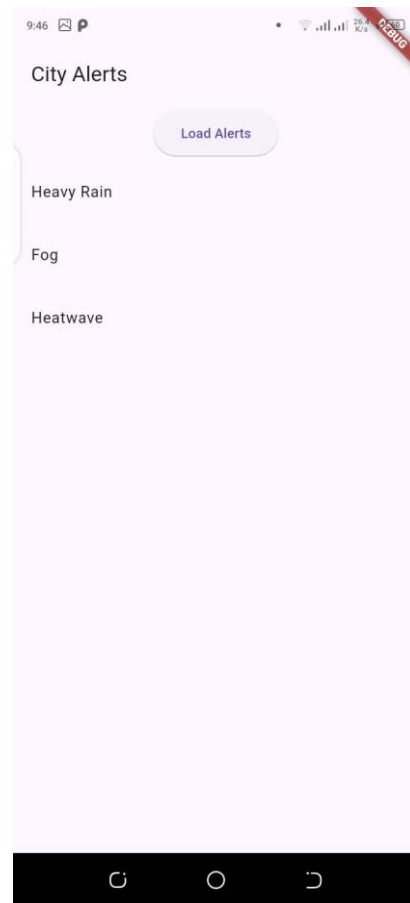
```
class CityAlertsScreen extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text("City Alerts")),  
      body: Column(  
        children: [  
          ElevatedButton(  
            onPressed: () {  
              context.read<AlertBloc>().add(FetchAlertsEvent());  
            },  
            child: const Text("Load Alerts"),  
          ),  
          Expanded(  
            child: BlocBuilder<AlertBloc, AlertState>(  
              builder: (context, state) {  
                if (state is AlertLoading) {  
                  return const Center(child: CircularProgressIndicator());  
                }  
              },  
            ),  
          ),  
        ],  
      ),  
    );  
  }  
}
```

```
    } else if (state is AlertLoaded) {  
      return ListView.builder(  
        itemCount: state.alerts.length,  
        itemBuilder: (context, index) {  
          return ListTile(  
            title: Text(state.alerts[index].title),  
          );  
        },  
      );  
    }  
    return const Center(child: Text("No alerts loaded"));  
  },  
),  
),  
],  
),  
);  
}  
}
```

1.3 Screenshots



App Screen



Alerts Loaded

